Chapter 13 Warm-Up: The Diamond Inheritance Problem

Prof. Jordan C. Hanson

October 28, 2026

1 A Diamond-Shaped Class Hierarchy

Suppose we begin with a general `Student` class:

```
class Student:
    def describe(self):
        return "Student"
```

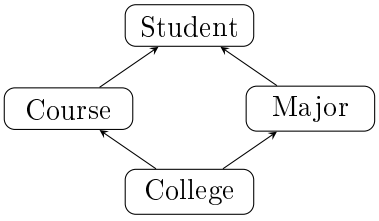
Now create two child classes:

```
class Course(Student):
    def describe(self):
        return "Course group"
class Major(Student):
    def describe(self):
        return "Major group"
```

Both `Course` and `Major` inherit from `Student`, but each **overrides** the same method.
Finally, define

```
class College(Course, Major):
    pass
```

The inheritance pattern forms a diamond:



Question: Why might the method `describe()` appear ambiguous for a `College` object?

2 Which Method Runs?

Create an object:

```
wc = College()
print(wc.describe())
```

Before running the code, predict the output. There are two possible inherited versions:

```
Course.describe()
Major.describe()
```

Python must decide which one has priority.

3 Method Resolution Order

Python resolves multiple inheritance using the **Method Resolution Order**, or **MRO**.
Try:

```
print(College.mro())
```

For

```
class College(Course, Major):
```

Python searches classes according to the MRO.
Which class is checked first after `College`? Which class is checked next? Therefore, which version of `describe()` should run?

4 Reverse the Parent Order

Now change the class definition to

```
class College(Major, Course):
    pass
```

Predict:

```
wc = College()
print(wc.describe())
```

What changed?

Key idea: In multiple inheritance, Python uses the MRO to decide which inherited method to call when more than one parent provides a method with the same name.